MoleculeGraph

Sistema di rappresentazione e manipolazione di molecole chimiche tramite grafi

Relazione di Progetto

Modulo di Linguaggio C — ITS A.A. 2025/2026

Componenti del gruppo:

Cavalieri Nicolò
Comelli Riccardo Boris
Covino Edo
Gigliotti Gabriel
Zanchini Mattia

1. Struttura dati scelta e motivazione

1.1 Rappresentazione del grafo

Una molecola è modellata come un **grafo non orientato**: gli atomi corrispondono ai nodi e i legami chimici agli archi. Per rappresentare questa struttura in C abbiamo scelto un approccio basato su **array statici**, che consiste in due array all'interno di una struttura principale Molecule: un array di nodi (Atom) e un array di archi (Bond).

Campo	Tipo	Descrizione
atoms[]	Atom[MAX_ATOMS]	Array dei nodi del grafo (gli atomi)
bonds[]	Bond[MAX_ATOMS²]	Array degli archi del grafo (i legami)
atom_count	int	Numero di atomi attualmente presenti
bond_count	int	Numero di legami attualmente presenti
next_id	int	Prossimo ID univoco da assegnare

Ogni nodo Atom contiene: l'identificatore univoco id, il simbolo chimico (es. "O", "H"), la valenza massima max_valence e la valenza attualmente occupata used_valence. Ogni arco Bond contiene gli ID dei due atomi coinvolti e il tipo di legame (1=singolo, 2=doppio, 3=triplo).

1.2 Motivazione della scelta

Abbiamo preferito array statici rispetto a una lista di adiacenza dinamica per le seguenti ragioni:

- Il numero massimo di atomi (MAX_ATOMS = 64) è noto e limitato: non è necessaria allocazione dinamica.
- L'accesso diretto per indice rende semplice e veloce trovare un atomo dato il suo ID.
- La gestione della memoria è completamente automatica: nessun malloc/free, nessun rischio di memory leak.
- La semplicità della struttura facilita il debug e la comprensione del codice.

Lo svantaggio principale è il limite fisso di MAX_ATOMS atomi, ma per la scala di questo progetto (molecole semplici) è una limitazione accettabile.

2. Funzioni implementate

F1 — add_atom

Aggiunge un nuovo atomo alla molecola assegnandogli un ID univoco crescente (next_id). Prima di inserire l'atomo verifica che non sia stato raggiunto il limite MAX_ATOMS e che il simbolo chimico sia tra quelli supportati (H, O, C, N). La valenza massima viene ricavata automaticamente dal simbolo tramite la funzione helper interna get_max_valence. Ritorna l'ID assegnato oppure -1 in caso di errore.

F2 — add_bond

Aggiunge un legame tra due atomi esistenti. Esegue tre controlli in sequenza: verifica che entrambi gli ID esistano nella molecola, controlla che non esista già un legame tra quegli stessi atomi, e verifica che la valenza di entrambi gli atomi possa sostenere il nuovo legame (used_valence + bond_type <= max_valence). Se uno qualunque dei controlli fallisce, stampa un messaggio di errore su stderr e ritorna -1 senza modificare la struttura. La principale difficoltà è stata gestire correttamente il controllo della valenza per i legami doppi e tripli, che consumano più di una unità di valenza per ciascun atomo.

F3 — remove_atom

Rimuove un atomo e tutti i legami ad esso collegati. La parte più delicata è la compattazione degli array dopo la rimozione: per i legami, si scorre l'array dall'inizio e ogni volta che si trova un legame che coinvolge l'ID da rimuovere si decrementa la used_valence dell'altro atomo e si compatta l'array shiftando gli elementi successivi. La stessa operazione di shift viene poi applicata all'array degli atomi. La difficoltà principale è stata gestire correttamente l'indice di scorrimento durante la compattazione, per evitare di saltare elementi.

F4 — get_formula

Calcola la formula bruta della molecola. Scorre l'array degli atomi contando quante occorrenze ci sono di ciascun elemento, poi costruisce la stringa in ordine alfabetico

(C, H, N, O) usando sprintf e strcat. Il numero viene omesso quando il contatore è 1 (es. "H2O" non "H2O1"). Non ha presentato difficoltà particolari, ma richiede attenzione all'ordine degli elementi e alla gestione del buffer di output.

F5 — merge_molecules

Fonde due molecole in una nuova creando un legame tra un atomo della prima e uno della seconda. La parte più complessa è la rinumerazione degli ID: gli atomi della prima molecola mantengono i loro ID originali, mentre gli atomi della seconda vengono rinumerati aggiungendo un offset pari al next_id della prima molecola. Questo garantisce che non ci siano collisioni di ID nella molecola risultante. La stessa trasformazione viene applicata agli ID nei legami della seconda molecola prima di copiarli. Infine viene aggiunto il legame di giunzione tramite add_bond.

3. Documentazione dell'utilizzo dell'LLM

3.1 Struttura dati: motivazione e confronto

Abbiamo utilizzato Claude per capire come rappresentare un grafo in C prima di scegliere l'approccio implementativo.

Prompt inviato:

"Come si rappresenta un grafo non orientato in C? Quali sono le differenze tra lista di adiacenza e array di archi, e quando conviene usare uno rispetto all'altro?"

Risposta ricevuta (sintesi):

L'LLM ha spiegato le due rappresentazioni principali: la lista di adiacenza, dove ogni nodo ha una lista dei suoi vicini, è efficiente per grafi sparsi e per operazioni come "trovare tutti i vicini di un nodo". L'array di archi (edge list), dove si mantiene semplicemente un array di coppie (nodo1, nodo2), è più semplice da implementare e sufficiente per grafi piccoli e densi. Per grafi con al massimo qualche decina di nodi e operazioni prevalentemente di inserimento e ricerca, l'array di archi è la scelta più semplice e diretta.

Cosa abbiamo capito e modificato:

Abbiamo scelto l'array di archi perché le molecole nel progetto hanno pochi atomi (MAX_ATOMS = 64) e le operazioni principali sono aggiunta, ricerca e rimozione di legami — tutte gestibili efficientemente con un array. Abbiamo però aggiunto rispetto alla risposta dell'LLM il campo used_valence direttamente nella struttura Atom, invece di ricalcolarlo ogni volta scorrendo i legami: questo semplifica notevolmente il controllo della valenza in add_bond.

3.2 Gestione della compattazione in remove_atom

Prompt inviato:

"In C, come si rimuove un elemento da un array statico senza lasciare buchi? Devo rimuovere sia un elemento dall'array degli atomi che tutti i legami che lo coinvolgono dall'array dei legami."

Risposta ricevuta (sintesi):

L'LLM ha suggerito la tecnica dello shift: quando si trova l'elemento da rimuovere all'indice i, si spostano tutti gli elementi successivi di una posizione verso sinistra con un ciclo for (arr[j] = arr[j+1]), poi si decrementa il contatore. Ha anche avvertito di fare attenzione all'indice di scorrimento quando si rimuovono più elementi in un unico passaggio: se si rimuove l'elemento in posizione i e si shifta, il prossimo elemento è ora in posizione i e non va incrementato l'indice.

Cosa abbiamo capito e modificato:

Abbiamo seguito il suggerimento per la logica di shift, ma abbiamo dovuto adattarlo perché nel nostro caso occorre rimuovere più legami in un unico passaggio (tutti quelli che coinvolgono l'atomo rimosso). Abbiamo usato un ciclo while invece di un for, senza incrementare l'indice quando trovo e rimuovo un legame, esattamente come avvertito dall'LLM. Abbiamo anche aggiunto il decremento di used_valence sull'altro atomo coinvolto nel legame, che la risposta generica dell'LLM non contemplava perché non conosceva la nostra struttura dati specifica.

3.3 Riflessione complessiva

L'LLM è stato utile soprattutto nella fase iniziale di progettazione, per confrontare approcci diversi e scegliere quello più adatto al nostro caso. Le risposte erano corrette a livello teorico ma sempre generiche: non conoscevano il contesto specifico del progetto (la valenza degli atomi, la struttura Bond, il campo used_valence). Ogni volta abbiamo dovuto adattare i concetti spiegati alla mia implementazione concreta, il che ha richiesto una comprensione reale di quello che stavamo facendo. Non sarebbe stato possibile usare il codice generato direttamente senza capirlo, perché non avrebbe mai corrisposto esattamente alle strutture che avevamo definito.

4. Output del programma

Di seguito l'output prodotto dal programma per ciascuno dei 5 scenari di test richiesti dalla traccia.

Scenario 1 — O2

```
=== Scenario 1: O2 ===
--- Molecola (2 atomi, 1 legami) ---
  Atomo [0] O  (valenza: 2/2)
  Atomo [1] O  (valenza: 2/2)
  Legame 0--1  tipo: 2
Formula: O2  (attesa: O2)
```

Scenario 2 — H2O

```
=== Scenario 2: H2O ===
--- Molecola (3 atomi, 2 legami) ---
  Atomo [0] O  (valenza: 2/2)
  Atomo [1] H  (valenza: 1/1)
  Atomo [2] H  (valenza: 1/1)
  Legame 0--1  tipo: 1
  Legame 0--2  tipo: 1
Formula: H2O  (attesa: H2O)
```

Scenario 3 — Violazione di valenza

```
=== Scenario 3: violazione valenza ===
Errore: valenza dell'atomo O [0] superata (2/2).
Legame rifiutato correttamente: valenza dell'ossigeno gia' satura.
```

Scenario 4 — H2O2 tramite merge

```
=== Scenario 4: H2O2 via merge ===
--- Molecola (4 atomi, 3 legami) ---
  Atomo [0] O  (valenza: 2/2)
  Atomo [1] H  (valenza: 1/1)
  Atomo [2] O  (valenza: 2/2)
  Atomo [3] H  (valenza: 1/1)
  Legame 0--1  tipo: 1
  Legame 2--3  tipo: 1
  Legame 0--2  tipo: 1
Formula: H2O2  (attesa: H2O2)
```

Scenario 5 — Rimozione di un H da H2O2

```
=== Scenario 5: rimozione atomo da H2O2 ===
--- Molecola (3 atomi, 2 legami) ---
  Atomo [0] O  (valenza: 1/2)
  Atomo [2] O  (valenza: 2/2)
  Atomo [3] H  (valenza: 1/1)
  Legame 2--3  tipo: 1
  Legame 0--2  tipo: 1
Formula: HO2  (attesa: HO2)
```